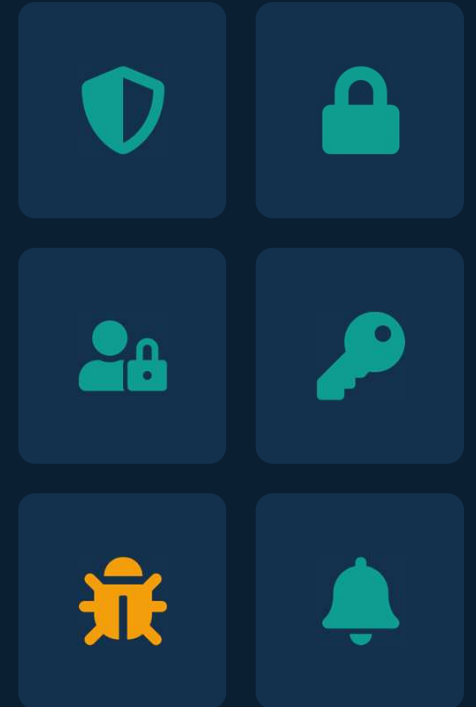


FIL FRESHER BATCH · GURGAON

Secure Coding Practices

OWASP Top 10:2025 for banking applications, in Python and Java

Case study: PBMS, the Personal Banking Management System · hands-on lab included



Facilitator: Abinash Kumar Mishra

Session plan

Concepts in the morning, hands-on in the afternoon

09:30 ● Why secure coding matters in banking

09:45 ● OWASP Top 10:2025, part 1: A01 – A05

live demos

11:15 ● OWASP Top 10:2025, part 2: A06 – A10

live demos

12:30 ● Beyond the Top 10: secrets, IAM, DevSecOps, cloud, AI

14:00 ● PBMS Secure Coding Lab (pairs)

150 min

16:30 ● Lab review, scoreboard & takeaways

30 min

Breaks

- 11:00 – 11:15 Tea
- 13:15 – 14:00 Lunch
- 15:30 – 15:40 Tea

You will need

- Python 3.10+
- VS Code
- The lab zip
- A partner

Why secure coding matters in finance



Trust is the product

Customers hand us their money and their data. One breach can undo decades of reputation.



Attackers follow the money

Financial services are among the most targeted industries, and attacks are automated and continuous.



Regulators are watching

RBI / SEBI cyber-security frameworks, India's DPDP Act 2023, GDPR and PCI DSS all carry penalties and audits.



It starts with your code

Most breaches exploit ordinary bugs: a missing check, a string-built query, a secret in a log.

OWASP Top 10:2025 at a glance

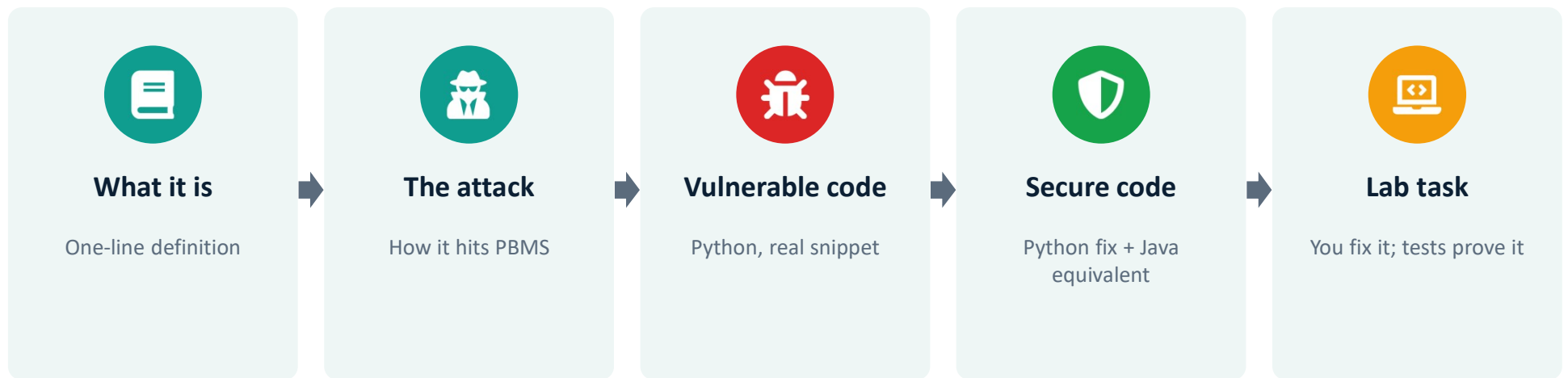
The industry's consensus list of the most critical web application risks

A01	Broken Access Control	<i>#1 since 2021</i>	A06	Insecure Design	<i>design, not a bug</i>
A02	Security Misconfiguration	<i>up from #5</i>	A07	Authentication Failures	<i>identity & sessions</i>
A03	Software Supply Chain Failures	<i>new & expanded in 2025</i>	A08	Software or Data Integrity Failures	<i>trust without verification</i>
A04	Cryptographic Failures	<i>was #2 in 2021</i>	A09	Security Logging & Alerting Failures	<i>renamed: now 'alerting'</i>
A05	Injection	<i>#3 in 2021</i>	A10	Mishandling of Exceptional Conditions	<i>new in 2025</i>

What changed from 2021: Supply Chain (A03) and Exceptional Conditions (A10) are new; SSRF is now part of A01; Misconfiguration rose to #2.

How each topic is taught today

Same five steps for every risk, and the same banking app throughout



PBMS: a FastAPI banking API with customers (alice, bob), an admin, accounts, transaction search, fund transfers and statement import. Every code sample today comes from it, and the afternoon lab is the same codebase.

01



OWASP Top 10:2025 · A01 – A05

09:45 – 11:00

BY THE END OF THIS MODULE YOU CAN



Recognise A01–A05 in real banking code



Exploit each one on PBMS in a live demo



Apply the standard fix in Python and Java



A01: Broken Access Control

A01 · OWASP 2025

Users can act outside their intended permissions: read another customer's data, call admin functions, or change records they don't own. In 2025 SSRF is folded into this category.



ATTACK ON PBMS

Bob logs in, opens `/api/accounts/201`, then changes the URL to `/api/accounts/101` and sees Alice's balance. This is called IDOR (Insecure Direct Object Reference).

How to defend

- ✓ Deny by default; check authorization on the server for every request
- ✓ Check object ownership, not just "is logged in"
- ✓ Enforce roles server-side; never trust client headers like X-Role
- ✓ Return 403/404 consistently and log access denials



In the lab: `TODO[A01-1..3]` · `app/deps.py`, `app/routers/accounts.py`

A01 in code: check the owner every time

A01 · CODE

VULNERABLE

python

```
@router.get("/accounts/{account_id}")
def get_account(account_id: int,
                user=Depends(get_current_user)):
    row = conn.execute(
        "SELECT * FROM accounts WHERE id = ?",
        (account_id,)).fetchone()
    # BAD: any logged-in user can read ANY account
    return to_dict(row)
```

SECURE

python

```
def get_owned_account(conn, account_id, user):
    row = conn.execute(
        "SELECT * FROM accounts WHERE id = ?",
        (account_id,)).fetchone()
    if row is None:
        raise HTTPException(404, "Account not found")
    if row["owner_id"] != user.id \
        and user.role != "admin":
        raise HTTPException(403, "No access")
    return row
```




A02: Security Misconfiguration

A02 · OWASP 2025

Insecure defaults, debug mode in production, verbose errors, default credentials, open CORS, missing security headers, unnecessary features switched on.



ATTACK ON PBMS

PBMS goes live with debug=True. An attacker triggers an error and receives a stack trace containing file paths, library versions and a database host name. That is a map for the next attack.

How to defend

- ✓ Secure defaults: debug off unless explicitly enabled
- ✓ Fail fast at start-up if prod config is unsafe
- ✓ Secrets from env / vault; no hard-coded fallback keys
- ✓ Explicit CORS allow-list; security headers on every response



In the lab: TODO[A02-1..4] · app/config.py, app/main.py

A02 in code: refuse to start with unsafe config

A02 · CODE

VULNERABLE

```
python
app = FastAPI(debug=True) # BAD: traces to users

SECRET_KEY = os.getenv("KEY", "changeme") # BAD

app.add_middleware(CORSMiddleware,
                    allow_origins=["*"]) # BAD
```

SECURE

```
python
debug = env.get("APP_DEBUG", "false") == "true"
if env_name == "prod" and debug:
    raise ConfigError("debug must be off in prod")
secret = env.get("APP_SECRET_KEY", "")
if not secret and env_name == "prod":
    raise ConfigError("APP_SECRET_KEY is required")
if "*" in origins:
    raise ConfigError("wildcard CORS not allowed")
# middleware adds: nosniff, DENY, no-store
```



A03: Software Supply Chain Failures

A03 · OWASP 2025

A dependency, build step or artifact is compromised before it reaches production: typo-squatted packages, dependency confusion, poisoned libraries, tampered images, compromised CI runners.



ATTACK ON PBMS

A developer types `pip install requets` (missing 's'). The typo-squatted package installs fine, works like requests, and quietly sends environment variables (cloud keys, DB passwords) to the attacker.

How to defend

- ✓ Pin exact versions; use a lock file with hashes
- ✓ Install only from the internal mirror / approved list
- ✓ Scan dependencies (SCA) on every PR; generate an SBOM
- ✓ Protect the build: least-privilege CI, signed artifacts



In the lab: `TODO[A03-1..2]` · `requirements.txt`

A03 in code: pinned, approved, verified

A03 · CODE

VULNERABLE

python

```
# requirements.txt
fastapi          # BAD: whatever is latest
uvicorn>=0.20    # BAD: a range, not a pin
requests         # BAD: typo-squatted!
colorama         # BAD: not approved
```

SECURE

python

```
# requirements.txt - exact, approved versions
fastapi==0.141.1
uvicorn==0.53.0
httpx==0.28.1
pydantic==2.13.3

$ pip-compile --generate-hashes -o requirements.lock
$ pip install --require-hashes -r requirements.lock
$ pip-audit -r requirements.txt # CVEs
```



A04: Cryptographic Failures

A04 · OWASP 2025

Sensitive data exposed through missing or weak cryptography: plain-text or fast-hashed passwords, weak algorithms (MD5, SHA-1, DES), hard-coded keys, no TLS.



ATTACK ON PBMS

A database backup of PBMS leaks. Passwords are unsalted MD5, so the attacker cracks most of them in minutes with a GPU and reuses them on customers' email accounts.

How to defend

- ✓ Passwords: slow, salted hash (Argon2id, bcrypt, PBKDF2 $\geq 600k$)
- ✓ Compare secrets in constant time (hmac.compare_digest)
- ✓ TLS 1.2+ in transit; AES-GCM at rest with managed keys (KMS)
- ✓ Never invent your own crypto; never hard-code keys



In the lab: TODO[A04-1..2] · `app/security/passwords.py`

A04 in code: salted, slow, constant-time

A04 · CODE

VULNERABLE

python

```
def hash_password(password):
    # BAD: fast, unsalted - cracked in seconds
    return hashlib.md5(password.encode()).hexdigest()

def verify(password, stored):
    # BAD: == leaks timing information
    return hash_password(password) == stored
```

SECURE

python

```
ITERATIONS = 600_000
def hash_password(password: str) -> str:
    salt = os.urandom(16) # new salt each time
    dk = hashlib.pbkdf2_hmac("sha256",
                             password.encode(), salt, ITERATIONS)
    return (f"pbkdf2_sha256${ITERATIONS}"
            f"${b64(salt)}${b64(dk)}")

def verify_password(password, stored) -> bool:
    # parse salt + iterations, re-derive candidate
    return hmac.compare_digest(candidate, expected)
```



A05: Injection

A05 · OWASP 2025

Untrusted input is interpreted as part of a command or query: SQL, OS commands, LDAP/NoSQL, ORM/HQL, expression languages and templates.



ATTACK ON PBMS

The transaction search puts the text box value straight into SQL. Typing `x' OR 1=1 --` returns every customer's transactions; a UNION query can read the users table.

How to defend

- ✓ Parameterised queries / ORM bindings, always
- ✓ Validate input at the edge (length, type, allow-list pattern)
- ✓ Escape LIKE wildcards (%) and (_) so they match literally
- ✓ Least-privilege DB account; no string-built commands



In the lab: `TODO[A05-1..3]` · `app/routers/transactions.py`

A05 in code: data is never code

A05 · CODE

VULNERABLE

```
python
sql = (f"SELECT * FROM transactions "
      f"WHERE account_id = {account_id} "
      f"AND description LIKE '%{q}%'")
rows = conn.execute(sql).fetchall()

# q = "x' OR 1=1 --" -> every customer's rows
```

SECURE

```
python
sql = ("SELECT * FROM transactions "
      "WHERE account_id = ? "
      "AND description LIKE ? ESCAPE '\\\\'")
rows = conn.execute(
    sql, (account_id, f"%{escape_like(q)}%")).fetchall()

# validate at the edge
q: str = Query(max_length=40,
               pattern=r"^[A-Za-z0-9 .,%_-]+$")
```


Injection is a family, not just SQL

A05 · INJECTION FAMILY

Same root cause everywhere: untrusted input mixed into something that gets executed

SQL injection

Input changes a SQL query



Parameterised queries, ORM bindings

OS command injection

Input reaches a shell command



subprocess with an argument list,
never shell=True

NoSQL / LDAP injection

Input changes a query object or filter



Typed queries; escape LDAP filter
values

ORM / HQL injection

String-built HQL / JPQL / raw ORM SQL



Named parameters, never
concatenation

EL / OGNL injection

Input evaluated by a framework
expression engine



Never evaluate user input; patch
frameworks

Template injection (SSTI)

Input becomes template source



Pass input as data; autoescape;
sandbox

02



OWASP Top 10:2025 · A06 – A10

11:15 – 12:30

BY THE END OF THIS MODULE YOU CAN



Recognise A06–A10 in real banking code



Design business rules as security rules



Fail closed and log what matters



A06: Insecure Design

A06 · OWASP 2025

Missing or ineffective security controls in the design itself. Perfect code can't fix a design that never asked "how could this be abused?"



ATTACK ON PBMS

PBMS transfer API accepts amount = -50,000: the 'sender' gains money. No daily limit, no same-account check, and a double-click on 'Pay' sends the money twice.

How to defend

- ✓ Threat-model features before coding (STRIDE, abuse cases)
- ✓ Write business rules as tests: limits, sign, ownership
- ✓ Idempotency keys for payments; rate limits for sensitive flows
- ✓ Validate at the edge AND in the service layer



In the lab: `TODO[A06-1..3]` · `app/services/transfers.py`, `app/routers/transfers.py`

A06 in code: business rules are security rules

A06 · CODE

VULNERABLE

python

```
class TransferRequest(BaseModel):
    from_account_id: int
    to_account_number: str
    amount_paise: int      # BAD: -5000 accepted

# BAD: no per-transfer or daily limit,
#      no same-account check,
#      no idempotency -> double-click pays twice
```

SECURE

python

```
amount_paise: int = Field(gt=0, le=MAX_PER_TRANSFER)
idempotency_key: str = Field(min_length=8)

if already_processed(idempotency_key):
    return {"status": "duplicate"}
if to_acc["id"] == from_acc["id"]:
    raise TransferError("Same account not allowed")
if balance < amount:
    raise TransferError("Insufficient funds")
if sent_today + amount > DAILY_LIMIT:
    raise TransferError("Daily transfer limit exceeded")
```



A07: Authentication Failures

A07 · OWASP 2025

Weak login, session or token handling: user enumeration, no brute-force protection, weak or reused passwords, tokens that can be forged or never expire, missing MFA.



ATTACK ON PBMS

The login says "User not found" vs "Wrong password", so the attacker learns valid usernames, then tries 10,000 common passwords with no logout. The token is unsigned JSON, so role=customer becomes role=admin.

How to defend

- ✓ One generic error message for every login failure
- ✓ Lock out or throttle after repeated failures; add MFA
- ✓ Signed tokens with short expiry; verify signature + exp
- ✓ Use the framework (Spring Security, OAuth2/OIDC); don't hand-roll



In the lab: TODO[A07-1..3] · `app/security/tokens.py`, `app/routers/auth.py`

A07 in code: generic errors, logout, signed tokens

A07 · CODE

VULNERABLE

```
python

if user is None:
    raise HTTPException(401, "User not found")
if not verify_password(pw, user["pw_hash"]):
    raise HTTPException(401, "Wrong password")
# BAD: reveals valid usernames, no logout

token = b64(json.dumps({"sub": uid, "role": role}))
# BAD: unsigned + no expiry -> edit role to admin
```

SECURE

```
python

if user is None or is_locked(user) \
    or not verify_password(pw, user["pw_hash"]):
    record_failure(user)          # lock after 5 tries
    raise HTTPException(401, GENERIC_LOGIN_ERROR)

payload = {"sub": uid, "role": role, "exp": now + 900}
sig = hmac.new(key, payload_b64, sha256).digest()

# verify: signature first, then expiry
if not hmac.compare_digest(sig, expected):
    raise InvalidToken("bad signature")
```



A08: Software or Data Integrity Failures

A08 · OWASP 2025

Code or data is trusted without checking its integrity: insecure deserialization, unsigned updates or files, CI/CD pipelines that accept unverified artifacts.



ATTACK ON PBMS

A partner uploads a daily statement file that PBMS unpickles. A crafted pickle runs `os.system()` on the PBMS server the moment it is loaded: remote code execution.

How to defend

- ✓ Never deserialize untrusted data with pickle / Java serialization
- ✓ Use data-only formats (JSON) + strict schema validation
- ✓ Verify a signature (HMAC or digital) BEFORE parsing
- ✓ Sign and verify build artifacts and updates



In the lab: `TODO[A08-1..3]` · `app/services/statement_import.py`

A08 in code: verify first, then parse data only

A08 · CODE

VULNERABLE

python

```
import pickle, base64

def load_statement(blob: bytes):
    # BAD: no signature check, and pickle can
    #       execute attacker code via __reduce__
    return pickle.loads(base64.b64decode(blob))
```

SECURE

python

```
def load_statement(blob, signature, key):
    expected = hmac.new(key, blob, sha256).hexdigest()
    if not hmac.compare_digest(expected, signature):
        raise StatementError("Signature mismatch")
    data = json.loads(blob.decode("utf-8")) # data only
    # only [{date, description, amount_paise}]
    validate_schema(data)
    return data
```




A09: Security Logging & Alerting Failures

A09 · OWASP 2025

Attacks go unnoticed because security events aren't logged, logs lack context, nobody is alerted, or logs themselves leak secrets and personal data.



ATTACK ON PBMS

An attacker tries 50,000 passwords over a weekend. Nothing is logged, so nobody knows. Meanwhile the debug log prints full account numbers and passwords, so the log store becomes the breach.

How to defend

- ✓ Log security events: login success/failure, lockout, access denied
- ✓ Structured (JSON) logs with timestamp and user, sent to the SIEM
- ✓ Never log passwords, tokens, full account or card numbers
- ✓ Alert on patterns (e.g. many failures) and rehearse the response



In the lab: `TODO[A09-1..2]` · `app/logging_utils.py` (+ A07-3 in `auth.py`)

A09 in code: log events, not secrets

[A09 · CODE](#)

VULNERABLE

python

```
log.warning(f"Wrong password {password} for {user}")
# BAD: the password is now in the log store

try:
    authenticate(user, password)
except Exception:
    return False    # BAD: silent - no audit trail
```

SECURE

python

```
def log_security_event(event, level=INFO, **fields):
    record = {"event": event, "ts": int(time.time())}
    for k, v in fields.items():
        record[k] = "***" if k in SENSITIVE else v
    security_logger.log(level, json.dumps(record))

log_security_event("login_failed", WARNING, username=u)
log_security_event("account_locked", WARNING,
                  username=u)
mask_account_number("50100012345601")    # XXXXXXXXXXXX5601
```



A10: Mishandling of Exceptional Conditions

A10 · OWASP 2025

The application fails open, swallows errors, leaks internals or leaves data inconsistent when an exception, timeout or partial failure happens.



ATTACK ON PBMS

During a transfer the debit succeeds, then the core-banking credit call times out. The code catches the error, ignores it and returns "completed". The money has left Alice's account and never reached Bob.

How to defend

- ✓ Fail closed: on error, deny / abort, never report success
- ✓ Use transactions: all steps commit together or roll back
- ✓ Generic error to the user + error ID; details only in logs
- ✓ No empty catch blocks; handle timeouts and clean up resources



In the lab: `TODO[A10-1..2]` · `app/services/transfers.py`, `app/main.py`

A10 in code: fail closed and roll back

A10 · CODE

VULNERABLE

```
python

try:
    debit(from_acc, amount)      # autocommited
    credit(to_acc, amount)       # times out
except Exception:
    pass                         # BAD: fails OPEN
return {"status": "completed"}   # money vanished

# global handler - BAD: leaks internals
return JsonResponse(500, {"detail": str(exc)})
```

SECURE

```
python

try:
    conn.execute("BEGIN")
    debit(from_acc, amount)
    credit(to_acc, amount)
    conn.execute("COMMIT")
except Exception:
    conn.execute("ROLLBACK")
    log_security_event("transfer_failed", ERROR, ...)
    raise                          # fail CLOSED

# global handler -> {"detail": "Internal server error",
#                  "error_id": "<uuid>"}
```

03



Beyond the Top 10

12:30 – 13:15

BY THE END OF THIS MODULE YOU CAN



Keep secrets out of code, logs and prompts



Apply least privilege and Zero Trust



Know where security gates sit in the pipeline

Core defence mechanisms

BEYOND THE TOP 10

Six principles behind every fix you saw this morning



Validate input

At every trust boundary: type, length, format, allow-list.



Parameterise

Keep code and data apart: queries, commands, templates.



Encode output

Encode for the context (HTML, JSON, SQL, shell) to stop XSS and friends.



Defence in depth

Several independent layers: one failure is not a breach.



Least privilege

Users, services and DB accounts get only what they need.



Use the framework

Built-in auth, validation and escaping beat hand-rolled code.

Framework security reference: Spring Boot vs FastAPI

BEYOND THE TOP 10

Security control	Java (Spring Boot)	Python (FastAPI)
Input validation	Bean Validation: @Valid, @NotNull, @Size, @Pattern	Pydantic models, Field(...), Query(...) constraints
Parameterised queries	Spring Data JPA @Query with :params, JdbcTemplate ?	SQLAlchemy ORM / text() with bound params, sqlite3 ?
Authentication	Spring Security, OAuth2 Resource Server (JWT)	fastapi.security (OAuth2), OIDC via identity provider
Authorization	@PreAuthorize, method security	Dependencies (Depends) that check role and ownership
Password hashing	BCryptPasswordEncoder / Argon2PasswordEncoder	argon2-cffi, bcrypt, hashlib.pbkdf2_hmac
XSS / output encoding	Thymeleaf auto-escaping	Jinja2 with autoescape=True (APIs: return JSON)
Error handling	@RestControllerAdvice + ProblemDetail	@app.exception_handler + generic JSONResponse

Secrets management

BEYOND THE TOP 10

A secret in Git is a secret published, even after you delete it

```
# BAD
DB_PASSWORD = "Pbms@Prod#2026"
headers = {"x-api-key": "sk_live_9f8e..."}

# GOOD - injected at runtime, never in the repo
# (from vault / cloud secret manager)
db_password = os.environ["DB_PASSWORD"]
api_key = secrets_client.get_secret("pbms/partner-key")

# block leaks before they happen
$ gitleaks detect --source . # pre-commit + CI
```

- **Store** in a vault / cloud secret manager, not .env files in Git
- **Scope** each secret to one service, least privilege
- **Rotate** regularly, and immediately after any exposure
- **Prefer** short-lived credentials (IAM roles, workload identity)
- **Never** in code, logs, test data, tickets, chats or AI prompts

Leaked a secret? Rotate first, then clean history. Deleting the commit is not enough.

Identity, access and Zero Trust

BEYOND THE TOP 10



IAM

Every human and service has an identity; permissions attach to roles.



MFA

A second factor for all privileged and sensitive access.



Least privilege

Just enough access, just in time. Review and remove regularly.



Break-glass

Emergency access that is rare, logged, alarmed and reviewed.

Zero Trust

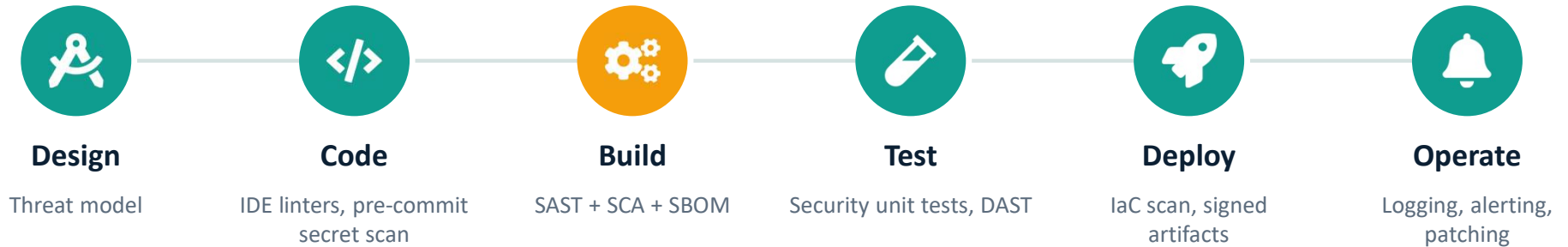
"Never trust, always verify."

- Verify every request: identity, device and context
- No implicit trust from being 'inside the network'
- Assume breach: segment, limit blast radius
- Continuously evaluate: sessions can be revoked
- In code: authorize on every call, as in A01

DevSecOps: security gates in the pipeline

BEYOND THE TOP 10

Shift left: find issues at the cheapest point, which is before merge



```
# .github/workflows/security.yml (illustrative - use your approved CI templates)
- run: gitleaks detect --source .           # secrets
- run: bandit -r app/                       # SAST for Python
- run: pip-audit -r requirements.txt        # SCA: known CVEs
- run: pytest -q                           # security tests (like today's lab)
# merge rule: no critical / high findings, all tests green
```

Cloud security fundamentals

The cloud provider secures the cloud; we secure what we put in it

BEYOND THE TOP 10

Shared responsibility

Provider

Data centres, hardware, hypervisor, managed-service internals

Us

Identities & IAM policies, network rules, data classification, encryption settings, app code, config, logging

Most common cloud breaches



Public storage buckets

Block public access by default (see the Terraform module lab)



Open ports

Security groups: no 0.0.0.0/0 on admin or DB ports



Over-permissive IAM

No wildcard * actions; roles per workload



No monitoring

Audit trails (e.g. CloudTrail) + alerts on anomalies



Unencrypted data

Encryption at rest (KMS) and in transit (TLS)

AI security: prompt injection and data leakage

BEYOND THE TOP 10

OWASP Top 10 for LLM Applications: LLM01 Prompt Injection, LLM02 Sensitive Information Disclosure

```
# BAD: secrets + raw customer data in the prompt,  
#      model output executed blindly  
prompt = f"""You are PBMS bot. DB password: {DB_PW}  
Customer: {customer_row}  
User: {user_message}""" # "ignore previous rules..."  
run_sql(llm(prompt))      # output treated as code  
  
# GOOD  
prompt = build_prompt(user_message,  
                      context=redact_pii(summary))  
reply = llm(prompt)  
if reply.tool not in ALLOWED_TOOLS: # allow-list  
    reject(reply)
```

- **Treat prompts as untrusted input.** User text can override system instructions.
- **Never put secrets in prompts.** Assume anything sent may be revealed.
- **Minimise data.** Classify and redact PII before it reaches a model.
- **Treat output as untrusted too.** Validate it; never execute it directly.
- **Least privilege for tools.** Allow-listed actions, scoped credentials, human approval for money movement.
- **Monitor.** Log prompts/outputs (redacted) and alert on abuse.

Quick check: spot the vulnerability

QUICK CHECK

2 minutes in pairs. Name the OWASP category and the fix.

1

```
email = request.args["email"]
sql = "SELECT * FROM users WHERE email = '" \
      + email + "'"
cursor.execute(sql)
```

2

```
@app.get("/statements/{sid}")
def get(sid: int, user=Depends(current_user)):
    return db.get_statement(sid)
```

3

```
logger.info(f"OTP {otp} sent to {phone}")
```

4

```
try:
    charge_card(order)
except Exception:
    pass
mark_order_paid(order)
```

04



Hands-on: the PBMS Lab

14:00 – 16:30

BY THE END OF THIS MODULE YOU CAN



Fix all ten OWASP 2025 risks in one real codebase



Prove each fix with automated tests



Score 100/100 on the grader

The lab project: pbms-secure-lab

A working FastAPI banking API, full of the bugs you saw this morning

LAB

```
pbms-secure-lab/
├── app/
│   ├── config.py           # A02 settings & secure defaults
│   ├── main.py             # A02 headers, A10 error handler
│   ├── deps.py             # A01 ownership & role checks
│   ├── logging_utils.py    # A09 masking & security events
│   ├── security/
│   │   ├── passwords.py    # A04 PBKDF2 hashing
│   │   └── tokens.py       # A07 signed, expiring tokens
│   ├── routers/            # auth, accounts, A05 search
│   └── services/
│       ├── transfers.py    # A06 business rules, A10 rollback
│       └── statement_import.py # A08
├── requirements.txt        # A03 pin & approve dependencies
├── supply_chain/approved-packages.txt
├── tests/                  # 69 tests (a01 ... a10)
└── grade.py                # scoreboard per OWASP category
```



Find

Search for TODO[A05 to find every task for a category.



Fix

Replace lines marked # INSECURE with secure code.



Prove

Run python grade.py a05 until it shows 10/10.

What a TODO looks like, and how you prove it

app/deps.py (starter)

```
def get_owned_account(conn, account_id, user):
    """Load an account the current user is allowed to see.

    TODO[A01-1]: Object-level authorization (prevents IDOR).
    * unknown account -> 404
    * account owned by someone else -> 403 (admins may read any)
    """
    row = conn.execute("SELECT * FROM accounts WHERE id = ?",
                        (account_id,)).fetchone()

    if row is None:
        raise HTTPException(404, "Account not found")
    return row          # <- your fix goes above this line
```

terminal

```
$ python grade.py a01
..F.FFF.
=====
A01 Broken Access Control      4/8      5.0
=====

# ...after your fix
$ python grade.py a01
.....
A01 Broken Access Control      8/8     10.0
```

- **Rules:** don't edit tests/ · don't rename functions · one category at a time · commit after each green category
- **Stuck?** read the test first. The assert tells you exactly what is expected.

Lab task map: 10 categories, 100 points

LAB

Cat.	What you implement	Where	Tests
A01	Ownership check (IDOR), list only my accounts, admin-only endpoint	deps.py, routers/accounts.py	8
A02	Debug off, required strong secret, no wildcard CORS, security headers	config.py, main.py	8
A03	Pin exact versions; only approved packages	requirements.txt	2
A04	PBKDF2 (600k) salted hashing; constant-time verify	security/passwords.py	5
A05	Parameterised search, LIKE escaping, input validation	routers/transactions.py	8
A06	Idempotency, amount limits, same-account, funds, daily limit	services/ & routers/transfers.py	10
A07	Signed expiring tokens, generic errors, logout after 5 failures	security/tokens.py, routers/auth.py	9
A08	HMAC-verify then JSON-parse statement; schema validation; no pickle	services/statement_import.py	10
A09	Account masking, JSON security events, secret redaction	logging_utils.py, routers/auth.py	6
A10	Transaction + rollback, fail closed, generic 500 with error_id	services/transfers.py, main.py	3

Each category is worth 10 points, scaled by the share of its tests you pass. Suggested order: A04 → A01 → A05 → A07 → A09 → A06 → A10 → A08 → A02 → A03.

LAB

 150 min

LAB

Run the lab

1

Unzip & set up `python -m venv .venv`, then `pip install -r requirements.txt`

2

Baseline score `python grade.py`: expect about 14/100

3

Read the student guide
`PBMS_Secure_Coding_Lab_Student_Guide.pdf` explains every task

4

Fix category by category `grade.py aNN` until 10/10, then commit

5

Try the attacks yourself `uvicorn app.main:create_app --factory`, then open `/docs`

6

Finish 100/100, then explain one fix to another pair

terminal

```
$ python -m venv .venv && source .venv/bin/activate
$ pip install -r requirements.txt
$ python grade.py
...
TOTAL                                     13.7/100

$ python grade.py a04
A04 Cryptographic Failures    1/5      2.0

# demo users: alice / Alice@Secure#2026
#                bob   / Bob@Secure#2026
```

Key takeaways



Never trust input

Validate at the edge; parameterise everything.



Authorize every call

Ownership and role, on the server, every time.



Crypto by the book

Slow salted hashes, signatures, managed keys.



Trust nothing unverified

Pinned dependencies, signed data, no pickle.



Log events, not secrets

Structured, masked, alerting to the SIEM.



Fail closed

Roll back, re-raise, generic errors to users.

You are the first line of defence: security is a quality attribute of every commit, not a final checkpoint.

References: owasp.org/Top10/2025 · OWASP Cheat Sheet Series · OWASP ASVS · OWASP Top 10 for LLM Applications